IoT Security Capstone

Experiment Results, Insights & Recommendations

Samantha Sweeney 07/19/26

01

The Threat: Unprotected IoT Pipelines

What happens when MQTT messages travel without protection?

Eavesdrop

An attacker with network access reads sensor data in plain text as it travels between device and broker, exposing operational details about the hotel's water systems.

Inject Fake Data

Without message signing, an attacker can send a fabricated reading that looks legitimate, triggering a false shutdown or masking a real problem.

Replay Attack

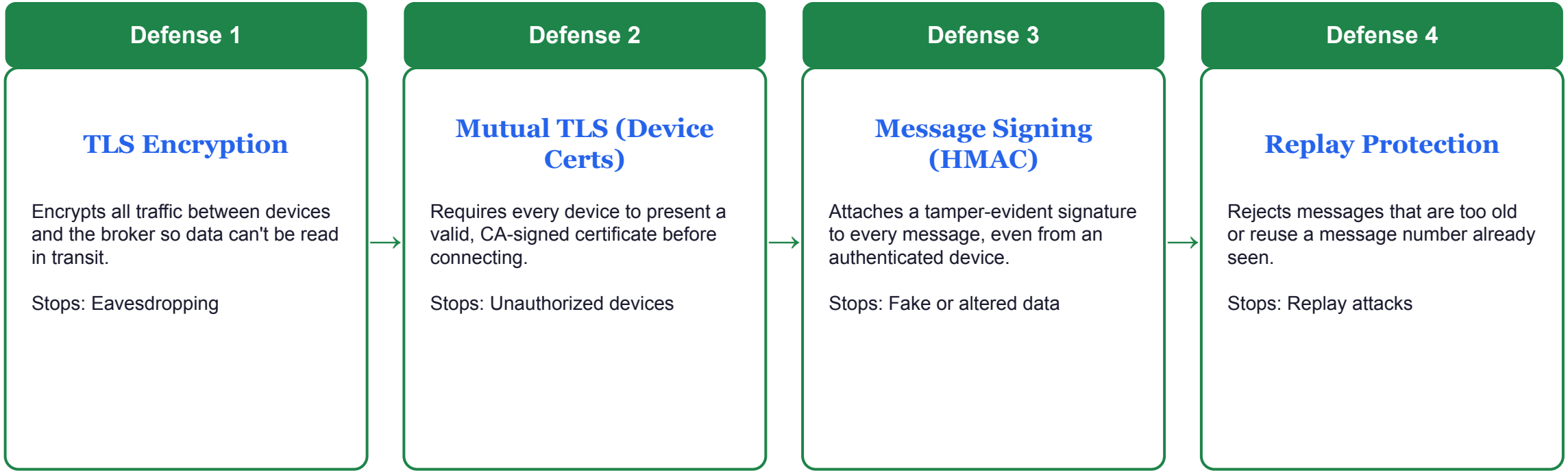
An attacker records a legitimate message and resends it later. The system accepts it as real because it's genuinely signed.

Unprotected pipelines are vulnerable because encryption alone verifies nothing about a message's origin, freshness, or integrity. Each gap has to be closed by its own defense.

02

Defense Architecture: Four Layers of Protection

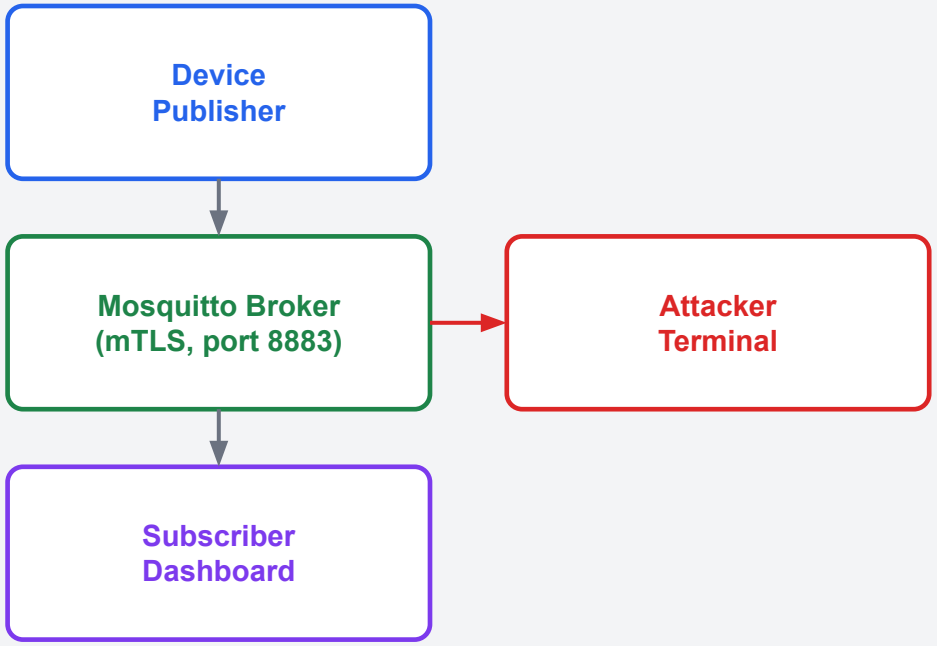
Each layer catches what the previous one can't.



You need all four because each layer defends against something the others can't. Encryption doesn't stop a compromised device from sending fake data. Verified identity doesn't stop a legitimate message from being replayed. Remove any one layer and its specific gap reopens.

03 Experiment Setup: Three-Phase Attack Simulator

A controlled test against the live pipeline to verify every defense works.



Your Setup Process

1. Started the Mosquitto broker configured for mTLS on port 8883, using CA-issued certificates for the broker and all three HYDROLOGIC devices.
2. Ran the defended publisher and subscriber scripts, layering TLS encryption, mutual TLS device authentication, HMAC message signing, and replay protection on top of the base MQTT pipeline.
3. Launched the attack simulator, which attempted an eavesdrop capture, a fake data injection, and a replay of a captured message against the live pipeline.
4. Monitored results through the terminal experiment summary and the live security dashboard, which flagged each rejected message in real time.

04 Attack Results: Every Attack Blocked

Which defense caught each attack phase — and why it works.

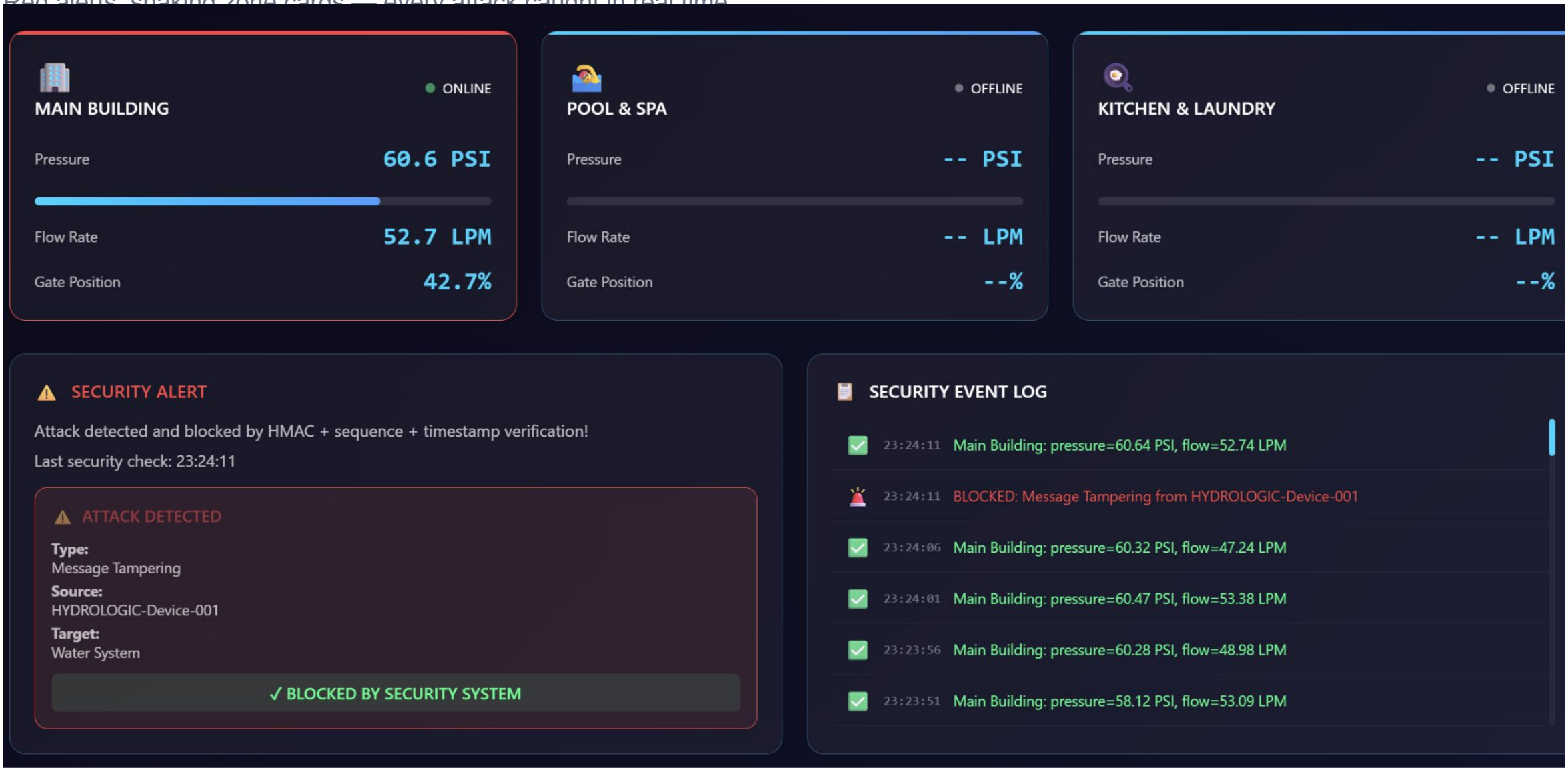
Attack Phase	What the Attacker Tried	Defense That Caught It	Why It Works
Phase 1: Eavesdrop	Captured traffic between a device and the broker	TLS Encryption	All traffic is encrypted in transit, so captured packets contain only ciphertext, not readable sensor data.
Phase 2: Inject	Sent a fabricated sensor reading with a valid-looking payload	HMAC Message Signing	Each message carries a signature computed from its contents; a forged message fails verification and is rejected.
Phase 3: Replay	Recorded a legitimate message and resent it later	Timestamp + Sequence Check	The replayed message reuses an old timestamp/sequence number already seen, so it's rejected as stale or duplicate.

✔ **Result: 0 attacks succeeded | 100% blocked**

05

Dashboard: Attack Detected

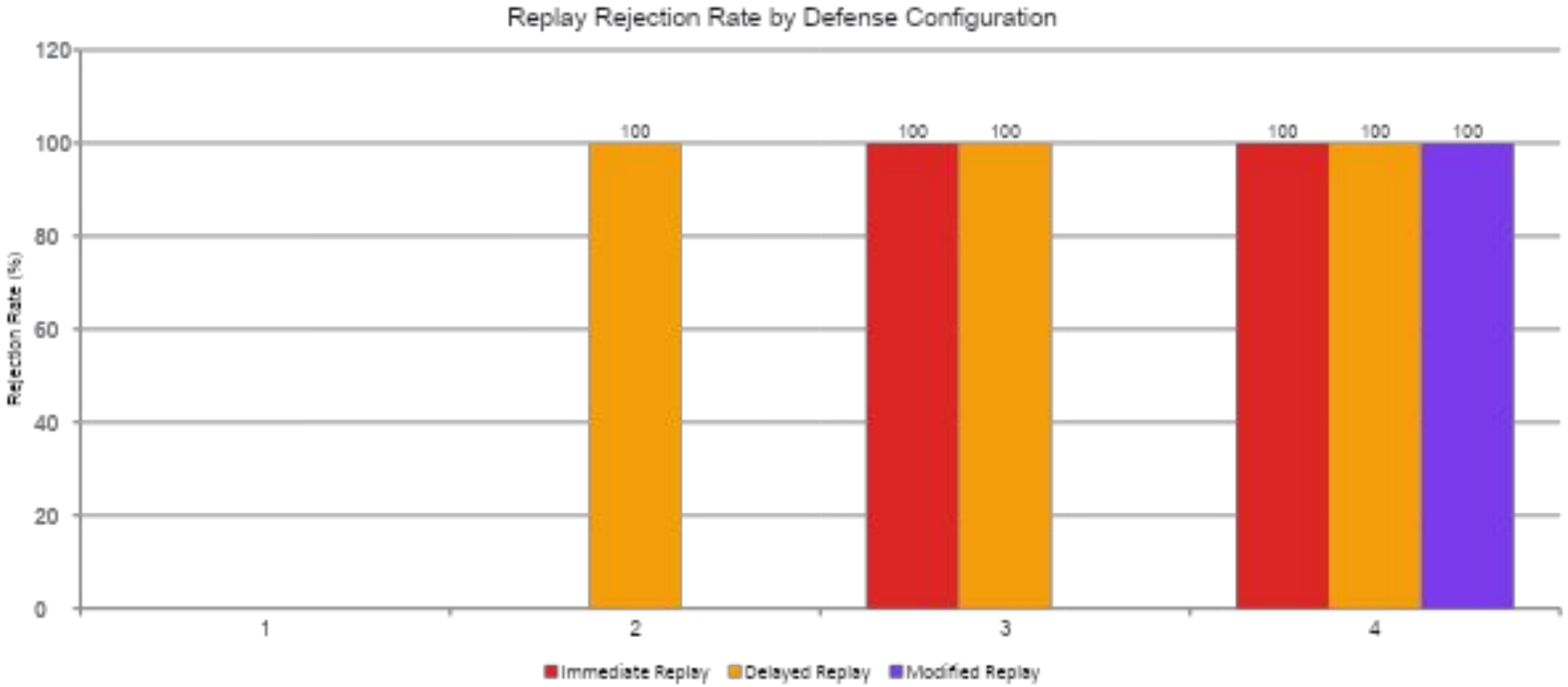
Red alerts, shaking zone cards — every attack caught in real time.



Bonus

Security Analysis Chart

Defense effectiveness across 4 configurations × 3 replay attack types (Project 6 experiment data).



06

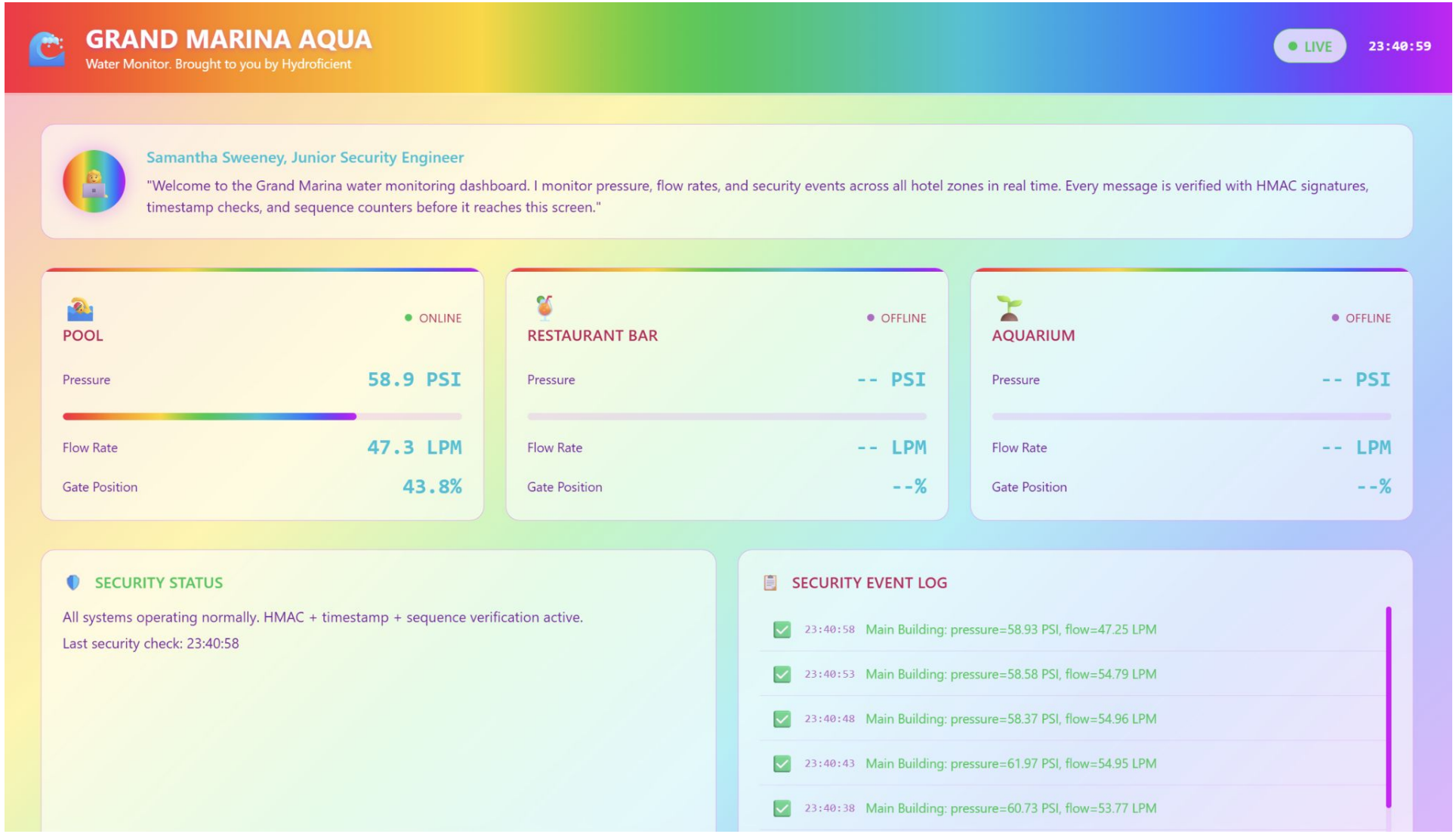
Stats: Attacks Blocked

The numbers tell the story — every attack rejected, every valid message accepted.



07 Custom Dashboard: Making It Mine

Personalized branding, AI-generated color theme, custom alert rules.



08 Key Insights

What this experiment revealed about IoT security.

1 No single defense is enough

mTLS stops outsiders but not compromised insiders. HMAC stops forgeries but not replays. You need all layers working together.

2 Security and performance weren't actually in tension

Across every layer tested, from mTLS connection overhead to the full replay defense stack, the added cost stayed under a few milliseconds, which is negligible against a 5-second reporting interval.

3 Most failures came from configuration, not cryptography

The hardest problems weren't the security algorithms themselves. They were mismatched CA keys, stale certificates, and a leftover background broker quietly accepting insecure connections.

4 Defense gaps surface through testing, not assumption

Running actual attack simulations, rather than reasoning abstractly about which defense stops what, is what revealed real gaps, like an HMAC-only setup missing replays, or a timestamp-only setup missing tampering.

09

Recommendations for Grand Marina

What should happen next to secure the IoT infrastructure at scale.

1

Deploy all four defense layers across every device

Replace the current publisher/subscriber scripts with the fully defended versions (mTLS + HMAC signing + replay protection) on all three HYDROLOGIC devices — partial deployment leaves the exact gaps shown in testing.

2

Formalize the Device Provisioning Policy

Require two-person verification for certificate installation and a 30-day certificate rotation cycle, so device identity stays trustworthy over time, not just at initial setup.

3

Add monitoring and alerting for rejected messages

A spike in rejected messages (failed signatures, replayed sequence numbers) is a strong signal of an active attack attempt and should trigger investigation, not silent rejection.

4

Audit for stray infrastructure before every deployment

This project's tests initially failed because a background broker process was silently accepting insecure connections — confirm only the intended, correctly configured broker is running before go-live.

Questions?